

Problem 1

The town of Thirtyville is preparing for a large expansion. Naturally, the mayor of Thirtyville needs to decide how the electrical network for the town should be designed. Let $G = (V, E)$ be a connected undirected graph with positive edge weights $w : E \rightarrow \mathbb{R}^+$ representing the expanded town. The vertices in V represent locations and the edges in E represent potential power lines that can be built, where the weight $w(e)$ of an edge $e \in E$ is the cost in dollars to build that power line.

The set of vertices V is composed of two disjoint sets $H \subset V$ and $P \subset V$; $V = H \cup P$. The vertices in H represent houses, and the vertices in P represent where power plants will be built. Describe an algorithm to compute the cheapest subset of power lines $L \subseteq E$ to build so that every house $h \in H$ is reachable from a power plant $p \in P$ via the power lines in L . Note that houses do not need to have a direct power line to a power plant; a path composed of power lines suffices. Justify the correctness of your algorithm. A formal proof is not necessary. Analyze the runtime. [Hint: Reduce the problem to computing a minimum spanning tree in a different graph. Include all parts of a reduction in your answer.]

Solution:

The algorithm uses G to create an auxiliary graph G' and finds the MST of G' . The MST is equivalent to the minimal cost subset of power lines L that connects all houses to power plants.

We create a new graph $G' = (V', E')$ by copying all vertices $v \in V$, $e \in E$, and edge weights $w(u, v) \in G$. Define a *supernode* p^* that connects to all power plants. Create an edge from p^* to all $p \in P$ with edge weight $w(p^*, p) = 0$.

We determine the MST in G' using an MST algorithm (Prim's, Kruskal's, Boruvka's). We now have a set of edges $L' \subseteq E'$ that spans all vertices $v' \in V'$ with minimal cost.

To transform the solution L' on G' back to G , we remove all edges $(p^*, p) \in L'$ for $p \in P$ that connect the supernode to the power plants. The remaining set of edges, $L = L' - \{(p^*, p)\}$, is the cheapest subset of power lines such that every house $h \in H$ is reachable from a power plant $p \in P$.

Correctness We will prove the following statement: There exists a minimum cost subset of power lines L in G that connects all houses to power plants with cost c_L if and only if there is an MST in G' whose weight is at most c_L .

$\Rightarrow$: If there exists a minimum-cost subset of power lines L in G that connects all houses to power plants, then there exists an MST in G' whose weight is at most the cost of L .

- Since L connects all $h \in H$ to all $p \in P$, L spans G .
- By definition, L is of minimal cost and does not contain any cycles. We can show the latter statement using the cycle property.
- So, L is a spanning tree for every vertex $v' \in V'$, except for the supernode p^* . We can use L to begin building an MST in G' .
- We can span p^* by adding an edge (p^*, p_i) to L , for any $p_i \in P$.
- Because $w(p^*, p_i) = 0$, adding these edges does not increase the total cost of L . This holds true for all $p_i \in P$.
- Therefore, the weight of the MST of G' is no greater than the cost of L .

$\Leftarrow$: If there exists an MST T in G' , then there exists a subset of power lines in G that connects all houses to power plants with cost at most the weight of T .

- By definition, T spans $V' = H \cup P \cup \{p^*\}$ with minimal cost.
- Define $T' = T - \{(p^*, p)\}$. In other words, T' contains the edges of T , excluding the edges which connect the supernode to the power plants.
- T' spans $H \cup P$ with minimal cost. Indeed, the cost of T' is the same as the cost of T because $w(p^*, p) = 0$.
- Therefore, T' is a feasible subset of power lines in G that connects all houses to power plants with cost no greater than the weight of T .

Runtime analysis

- Copying G takes $O(V + E)$ time
- Creating the supernode and connecting it to all power plants takes $O(1)$ and $O(P) = O(E)$ time, respectively.
- Finding an MST in $G' = (V', E')$ using Prim's algorithm takes time $O(E' \log V')$.
- Removing the edges connecting the supernode to the power plants takes time $O(P) = O(E)$.

Therefore, the total runtime is $O(E' \log V') = O(E \log V)$.

Problem 2

While you were solving the previous problem, the mayor has realized building power plants at the previously considered locations will be far too costly. Instead, they must solve the following problem (with the same setup as before, for completeness): Let $G = (V, E)$ be a connected undirected graph with positive edge weights $w : E \rightarrow \mathbb{R}^+$ representing the expanded town. The vertices in V represent locations and the edges in E represent potential power lines that can be built, where the weight $w(e)$ of an edge $e \in E$ is the cost in dollars to build that power line.

Now, a power plant can be built at **any location in V** for a **cost of z dollars**, where $z > 0$ is a given parameter (in this problem, there is no partition of V into different kinds of vertices). The goal is now to select locations $P \subseteq V$ to build power plants **and** which power lines $L \subseteq E$ to build so that **every** location in V is reachable from a power plant $p \in P$ via the power lines in L . Furthermore, the total cost of the power plants and power lines must be minimized.

For example, a feasible solution with cost $z|V|$ is to set $P = V$ and $L = \emptyset$, meaning that a power plant is built at every location, which is valid because every location is trivially reachable from itself with a power plant. For another example, a feasible solution is to build a single power plant (somewhere) and decide which edges to connect all locations to it. In between these extremes is the possibility to build more than one but less than $|V|$ power plants.

Describe an algorithm for this problem. Justify the correctness of your algorithm. A formal proof is not necessary. Analyze the runtime. [Hint: Reduce the problem to computing a minimum spanning tree in a different graph (perhaps similar, but not the same, as that for the previous problem). Include all parts of a reduction in your answer.]

Solution: Algorithm: We will use $G = (V, E)$ to create an auxiliary graph $G' = (V', E')$, such that the weight of the MST for G' is exactly the cost of the optimal way to open power plants and power lines.

G' consists of the original graph G with an extra vertex s that is joined to each vertex $v \in V$, with an edge weight of z .

$$V' = V \cup \{s\} \quad E' = E \cup \{\{s, v\} : v \in V\}$$

$$w'(u, v) = \begin{cases} w(u, v) & \text{if } \{u, v\} \in E \\ z & \text{if } \{u, v\} = \{s, v\} \text{ for some } v \in V \end{cases}$$

We then find the MST $T' = (V', E^*)$ for G' using Prim's algorithm. For each edge $\{s, v\}$ in E^* that has s as an endpoint, we open a powerplant at v . If S denotes the collection of such edges, we then open powerlines on each edge $\{u, v\}$ in $E^* \setminus S$.

Correctness:

Note: We give more details than necessary for full credit. Basically you should assert a correspondence with some justification.

- First, we show that any spanning tree $T' = (V', E^*)$ for G' corresponds to a valid solution $SOL = (P, L)$ with cost exactly the weight of T' .

For each edge $\{s, v\}$ in that has s as an endpoint, SOL has a powerplant at v with cost $z = w'(s, v)$. If S denotes the collection of such edges, we then open powerlines on each edge $\{u, v\}$ in $E^* \setminus S$, incurring cost $w'(u, v) = w(u, v)$ for each such edge. In other words,

$$P = \{v : \{s, v\} \in E^*\} \quad L = E^* \setminus S$$

The total cost of SOL is clearly the sum of edge weights in T' as desired. Moreover the solution is valid as the tree T' spans V' so every vertex $v \in V$ is connected to s , meaning it gets a power plant.

- Now, consider any a valid solution $SOL = (P, L)$ for the original problem. We will show that the cost of SOL is at least the cost of the MST on G' . We will build a collection of edges E^* that span G' as follows. For each $v \in P$, add an edge $\{s, v\}$ to E^* . The weight of each of these edges $w'(s, v)$ is exactly z , the cost of opening a power plant. Then add all the edges in L as well. Since $L \subset E$ the total weight of these edges is just the cost of opening the powerlines.

$$E^* = L \cup \{\{s, v\} : v \in P\}$$

The total weight of E^* is clearly the sum of the cost of opening the power plants in SOL and the cost of opening the power lines as desired. Moreover these edges put together must span G' as every $v \in V$ has a path to a power plant in P in the solution SOL , so there is a path from v to s in G' using only edges in E^* . Since every vertex is connected to s , there is a path between any two vertices in G' using only edges in E^* . Thus the cost of SOL must at least be the cost of an MST for G' .

Runtime analysis:

- Creating the graph G' takes $O(V + E)$ time as we just need to copy the original graph, and then add 1 vertex and $|V|$ many edges.
- Finding the MST for G' takes $O(E' + V' \log V') = O(E + V \log V)$ time using Prim's algorithm (when implemented using Fibonacci heaps), as $|V'| = |V| + 1$ and $|E'| = |E| + |V|$
- Constructing a solution from the MST T' takes $O(V' + E^*) = O(V)$ time, as $|V'| = |V| + 1$ and $|E^*| = |V'| - 1 = |V|$.
- Thus the time taken for Prim's algorithm dominates the running time, for a total running time of $O(E + V \log V)$.

Problem 3

Phoebe is an artisan potter that specializes in terracotta. She offers a vast catalog of vases with various sizes, colors, and shapes, and she has just received an immensely large and complicated order for n different designs of vases, $V_1, V_2, \dots, V_n$. Each design V_i takes a distinct number of minutes, s_i , for Phoebe to first sculpt by hand, and then another distinct number of minutes, f_i , for her to finish by firing it in her kiln. For purposes of this problem, suppose her kiln can fit any number of sculpted vases to fire and that it is OK for fully-fired vases to remain in the kiln until all vases are done firing.

Phoebe can only sculpt one design at a time, but immediately after she finishes sculpting a vase she adds it to the others in the kiln (if any) so that she can begin sculpting another vase. Once all vases are sculpted and fired in the kiln for at least as long as they all require, she pulls them all out at once and her job is done. Your goal is to help Phoebe determine which order she should sculpt the n designs in order to minimize the time when her job is done.

- (a) Prove that ordering the designs in increasing order of s_i is not optimal by providing a counterexample by providing an input for which the ordering does not yield the best possible end time.
- (b) Same as the last part, but with ordering in increasing order of $s_i + f_i$.
- (c) State an ordering for Phoebe to use and prove that it is optimal. You may find the following proof template useful¹:
 - (i) Compare your ordering to that of an arbitrary optimal ordering: If the orderings are not identical, then there are some pairs of designs in the optimal ordering that are out-of-order with respect to your ordering.
 - (ii) Consider the optimal ordering *with the fewest such pairs*.
 - (iii) Show that swapping two adjacent out-of-order designs in that ordering reduces the number of out-of-order pairs without increasing the overall end time.
 - (iv) Conclude that, because the resulting order has fewer out-of-order pairs and optimal end time, your ordering is indeed optimal by contradiction.

Solution:

- (a) We will prove this greedy policy is not optimal by counter-example.
Suppose $s_1 = 1, f_1 = 1, s_2 = 2, f_2 = 2$. Then, we sculpt V_1 before V_2 because $s_1 < s_2$.
The total sculpting and firing time in this ordering is: $1 + 2 + 2 = 5$.
However, if we sculpt V_2 before V_1 , then the total sculpting and firing time is: $2 + 2 = 4$.
This takes less time than the greedy choice, so the greedy choice is not optimal.
- (b) We will prove this greedy policy is not optimal by counter-example.
Suppose $s_1 = 1, f_1 = 1, s_1 + f_1 = 2, s_2 = 1, f_2 = 2, s_2 + f_2 = 3$. Then, we sculpt V_1 before V_2 .
The total sculpting and firing time in this ordering is: $1 + 1 + 2 = 4$.
However, if we sculpt V_2 before V_1 , then the total sculpting and firing time is: $1 + 2 = 3$.
This takes less time than the greedy choice, so the greedy choice is not optimal.

¹This is a *proof by smallest counterexample* instead of induction, which is functionally equivalent but perhaps more intuitive here. See the [notes on Induction](#) for a more in-depth discussion about this proof technique.

(c) **Informal Heuristic/Intuition:**

When cooking a meal, you can categorize dishes based on their prep times and cook times. If a dish needs to be cooked in an oven for a long time, you should probably prep the ingredients for that dish and put it in the oven first.

Ordering:

We will order the vase designs in **decreasing** order of the firing time f_i (note that there are no ties, due to distinctness), and sculpt them one after the other. This means that we first sculpt the vase with the largest f_i , and sculpt the vase with the smallest f_i at the end. Let the ordering obtained in this fashion be *ALG*.

Proof of Correctness:

For any arbitrary ordering O , a pair of designs (V_i, V_j) is said to be **out of order** with respect to *ALG* if the order they appear in O is not the same as *ALG*. Observe that if no pair is out of order, then both O and *ALG* must be the same ordering.

Consider an optimal ordering $OPT = (V_{i_1}, V_{i_2}, \dots, V_{i_n})$ of the designs which has the **fewest** out of order pairs with respect to *ALG*.

- If the firing time for V_{i_j} is **greater** than the firing time for $V_{i_{j+1}}$ for each $1 \leq j \leq n-1$, then the designs in OPT are in order of decreasing firing times, which is the same as *ALG* so our algorithm correctly computes the optimal ordering.
- If not, there exists some j such that the firing time for V_{i_j} is **lesser** than the firing time for $V_{i_{j+1}}$. In other words, we have 2 *adjacent* out of order pairs in OPT . We claim that swapping $(V_{i_j}, V_{i_{j+1}})$ produces a new ordering OPT^* that is at least as good as OPT , i.e., requires total time at most that of OPT . If this were the case, we have a **contradiction** as we have reduced the number of out of order pairs by exactly one, meaning that OPT wasn't the optimal solution with the fewest out of order pairs.

Firstly, note that the designs V_{i_k} for $k < j$ and $k > j+1$ still start and finish at the same time as they did before, as the total sculpting time for $(V_{i_j}, V_{i_{j+1}})$ after swapping remains the same

$$s_{i_{j+1}} + s_{i_j} = s_{i_j} + s_{i_{j+1}}$$

Let us suppose V_{i_j} was started at time t_{i_j} in the original ordering OPT . Then, in OPT the designs $(V_{i_j}, V_{i_{j+1}})$ get finished at times

$$t_{i_j} + s_{i_j} + f_{i_j}, \quad t_{i_j} + s_{i_j} + s_{i_{j+1}} + f_{i_{j+1}}$$

respectively. On the other hand, in OPT^* the design $V_{i_{j+1}}$ is done immediately before V_{i_j} , and they get finished at times

$$t_{i_j} + s_{i_{j+1}} + s_{i_j} + f_{i_j}, \quad t_{i_j} + s_{i_{j+1}} + f_{i_{j+1}}$$

respectively. Since $f_{i_j} < f_{i_{j+1}}$ (the pair was out of order), we have the inequalities

$$\begin{aligned} t_{i_j} + s_{i_{j+1}} + s_{i_j} + f_{i_j} &\leq t_{i_j} + s_{i_j} + s_{i_{j+1}} + f_{i_{j+1}} \\ t_{i_j} + s_{i_{j+1}} + f_{i_{j+1}} &\leq t_{i_j} + s_{i_j} + s_{i_{j+1}} + f_{i_{j+1}} \end{aligned}$$

where the RHS is the time at which design $V_{i_{j+1}}$ was finished in OPT . Thus, sculpting according to the ordering OPT^* takes total time at most that of OPT , which gives us a contradiction as discussed earlier.

Problem 4

Let X be a set of n intervals on the real line labeled $1, \dots, n$. We say that a subset of intervals $Y \subseteq X$ covers X if the union of all intervals in Y is equal to the union of all intervals in X . The *size* of a cover is just the number of intervals.

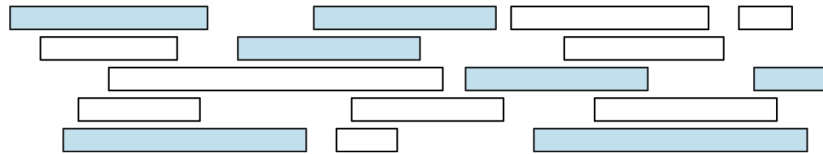


Figure 1. A set of intervals, with a cover (shaded) of size 7 (not necessarily the optimal cover).

Describe a greedy $O(n \log n)$ -time algorithm to compute the smallest cover of X . Assume that your input consists of two arrays $L[1..n]$ and $R[1..n]$, where $(L[i], R[i])$ is the i -th interval in X . Note that X as given is ordered arbitrarily. For simplicity, assume that no two endpoints of intervals are equal; that is, all elements of L, R are distinct. Prove the correctness of the algorithm (including an exchange argument) and analyze its runtime complexity.

Solution: We use a greedy algorithm that scans intervals from left to right. On encountering a left endpoint, if we have no chosen interval overlapping at this point, we must choose this interval. Otherwise, we search among all intervals with left endpoint before the right endpoint of the chosen interval overlapping this point and select such interval with farthest right endpoint.

```

COVER( $L, R, n$ ):
  Let  $Q$  be a queue of  $(L[i], R[i], i)$  tuples ordered from least to greatest  $L[i]$ 
  Let  $C$  be an empty list
   $r \leftarrow -\infty$ 
  while  $Q \neq \emptyset$ :
     $(\ell_i, r_i, i) \leftarrow Q.\text{remove}()$ 
    if  $\ell_i > r$ 
      while next tuple in  $Q$  has left endpoint at most  $r$ :
         $(\ell_j, r_j, j) \leftarrow Q.\text{remove}()$ 
        if  $r_j > r_i$ :
           $(\ell_i, r_i, i) \leftarrow (\ell_j, r_j, j)$ 
       $C.\text{append}(i)$ 
       $r \leftarrow r_i$ 
    else:
      continue
  return  $C$ 

```

Correctness. Let $x_1, x_2, \dots, x_m$ be the intervals chosen by the greedy algorithm sorted from least to greatest left endpoint. We will argue by induction that for all $1 \leq k \leq m$ there is an optimal schedule that chooses $x_1, x_2, \dots, x_k$.

For the base case, it is clearly necessary to choose the interval with the least left endpoint in order to cover the union of all intervals. For the inductive hypothesis (IH), suppose there is an optimal solution of the form $x_1, x_2, \dots, x_{k-1}, y_k, y_{k+1}, \dots, y_{m^*}$ where m^* is the number of intervals used in this optimal solution and again the intervals are ordered from least to greatest left endpoint. There are two cases.

1. If $L[x_k] > R[x_{k-1}]$ then $y_k = x_k$, otherwise the optimal solution does not cover all of x_k using only intervals with greater left endpoints.

2. Otherwise $L[x_k] < R[x_{k-1}]$, and we can further observe that $R[x_{k-1}] < R[x_k]$ from line 10 of the algorithm. In order to cover x_k , it must be that $L[y_k] < R[x_{k-1}]$. The algorithm chooses the interval with the greatest R , so $R[x_k] > R[y_k]$. Then $x_1, x_2, \dots, x_{k-1}, x_k, y_{k+1}, \dots, y_{m^*}$ must also be a valid optimal solution.

Runtime. The algorithm takes $O(n \log(n))$ to sort the n elements initially with, for example, mergesort. Thereafter, every iteration of the (nested) while loop takes constant time and removes an element from Q which initially has n elements, so a total of $O(n)$ time. The runtime is therefore dominated by the sorting, $O(n \log(n))$.